

Processor SDK Linux Automotive FAQ

Contents

Processor SDK Linux Automotive FAQ

- Topic wise FAQ pages
- Browsing code
- U-Boot
 - Where does the voltage configuration happen in MLO/U-boot?
 - How to change MPU OPP?
 - How to change the MPU VDD voltage?
 - Debug Only:** How to bypass the efuse (optimized) avs voltages and use hardcoded values?
 - How to change the CORE, DSPEVE, GPU, IVA VDD voltage?
 - Where does the clock configuration happen in MLO/U-boot?
 - How to change MPU/CPU frequency in MLO/u-boot?
 - How to change the frequency of remote cores (IVA,DSP,GPU) in kernel from u-boot?
 - How do I debug/ get more debug information for MMC related issues in u-boot?
- Kernel
 - How do I control MPU OPP (frequency and voltage)?
 - Add lower frequency OPP support for (<1GHz) J6Eco (dra72x) variants
 - How do I debug / get additional debug information for MMC/SD/SDIO issues?
 - How do I test uart data transfer?
 - Internal loop back within the controller
 - How do I measure time taken to execute code (timestamp) better than printk time?
- Multimedia
 - How do I fix the Yocto build issue while trying to build IPUMM component ?
 - How do I fix the issue with fetching IPUMM sources in the PSDKLA 3.03 or older installers
- Additional FAQs

Processor SDK Linux Automotive FAQ

Topic wise FAQ pages

The below topics have dedicated FAQ pages.

1. [Early Boot and Late Attach in Linux](#)
2. [Processor SDK - Linux Automotive Display FAQ](#)

Browsing code

One of the ways to browse through kernel/u-boot code is to use a utility like ctags or cscope or other text processing tools. On a Linux Host,

```
#sudo apt-get install ctags
#cd <source directory>
#ctags -R .
```

Takes a few minutes to index all symbols/functions.

You can browse by moving cursor to a symbols and then press "Ctrl+]" to find definition/declaration. If multiple definitions/declarations are available then the tool takes to first symbol - you can list all references by typing ":" or to the next by ":tn" or the previous by ":tp". Refer to "man ctags" for full details.

It's good to have atleast one code browsing utility installed.

U-Boot

Where does the voltage configuration happen in MLO/U-boot?

- v2016.05, v2017.01,

```
On power up PMIC configures all the VDDs as per the OTP programming/firmware on the PMIC.
The start point for voltage configuration is early_system_init() in arch/arm/cpu/armv7/omap-common/hwinit-common.c
```

Voltage configuration detailed flow

```
early_system_init() in arch/arm/cpu/armv7/omap-common/hwinit-common.c
vcores_update() in board/ti/dra7xx/evm.c (dra752_volts)
prcm_init() -> scale_vcores() in arch/arm/cpu/armv7/omap-common/clocks-common.c
```

How to change MPU OPP?

An OPP is a tuple of voltage and frequency. When changing OPP to higher always ensure to configure voltage first and then frequency. To change MPU OPP refer to [#How to change the MPU VDD voltage?](#) and [#How to change MPU/CPU frequency in MLO/u-boot?](#)

How to change the MPU VDD voltage?

Update the data structure in board/ti/dra7xx/evm.c 1. To modify existing voltage for OPP, such as OPP_NOM just update the value

```
struct vcores_data dra752_volts = {
    .mpu.value[OPP_NOM] = VDD_MPU_DRA7_NOM,
    .mpu.efuse.reg[OPP_NOM] = STD_FUSE_OPP_VMIN_MPU_NOM,
```

2. To add a new OPP add the entries and change current OPP

```
struct vcores_data dra752_volts = {
    .mpu.value[OPP_NOM] = VDD_MPU_DRA7_NOM,
    .mpu.efuse.reg[OPP_NOM] = STD_FUSE_OPP_VMIN_MPU_NOM,
    .mpu.value[OPP_HIGH] = VDD_MPU_DRA7_HIGH,
    .mpu.efuse.reg[OPP_HIGH] = STD_FUSE_OPP_VMIN_MPU_HIGH,
    .....
}
```

3. Change the opp in (u-boot versions < 2016.05 where CONFIG_*_OPP_* was not defined)
get_volttrail_opp() in board/ti/dra7xx/evm.c

```
case VOLT_MPU:
    opp = DRA7_MPU_HIGH;
in later versions , there's another macro whose value depends on CONFIG_xx options
case VOLT_MPU:
    opp = DRA7_MPU_OPP;
in arch/arm/include/asm/arch-omap5/clock.h, value of DRA7_MPU_OPP is defined based on config option selected
```

```
#if defined(CONFIG_DRA7_MPU_OPP_HIGH)
#define DRA7_MPU_OPP OPP_HIGH
#elif defined(CONFIG_DRA7_MPU_OPP_OD)
#define DRA7_MPU_OPP OPP_OD
#else /* OPP_NOM default */
#define DRA7_MPU_OPP OPP_NOM
#endif
```

Debug Only: How to bypass the efuse (optimized) avs voltages and use hardcoded values?

Disclaimer: Not recommended except for debug. DM recommends using AVS voltage at all times ofr all OPPs. Using non-optimized voltages will greatly reduce device life and Power on Hours

Below patch shows how to do this on v2014.07, should be applicable to 2016.05 and 2017.01 as well:

Disable avs in u-boot (<http://review.omapzoom.org/#/c/38381/>)

How to change the CORE, DSPEVE, GPU, IVA VDD voltage?

Refer to [#How to change the MPU VDD voltage?](#) section, just look for the VDD rail name that you want to configure.

Where does the clock configuration happen in MLO/U-boot?

- v2016.05, v2017.01,

The start point for clock configuration is prcm_init() in arch/arm/cpu/armv7/omap-common/hwinit-common.c

Clock configuration detailed flow

```
prcm_init() in arch/arm/cpu/armv7/omap-common/hwinit-common.c
setup_dpll() in arch/arm/cpu/armv7/omap-common/clocks-common.c
and
enable_basic_uboot_clocks() in arch/arm/cpu/armv7/omap4/hw_data.c
```

How to change MPU/CPU frequency in MLO/u-boot?

- V2016.05, v2017.01,

MPU_DPLL configuration is done in,

```
configure_mpu_dpll() in arch/arm/cpu/armv7/omap-common/clocks-common.c
```

and dpll_data structure comes from hw_data_init() in arch/arm/cpu/armv7/omap5/hw_data.c

for example for dra7xx, dra7xx_dpll in the same file holds the dpll data.

By default it's set to 1Ghz on dra7x, to change the frequency change the table entries or add a new table with required values

```
Details of the entries in the table
struct dpll_params {
    u32 m;
    u32 n;
    s8 m2;
    s8 m3;
    s8 m4;
    s8 m5;
    s8 m6;
};
/* OPP NOM FREQUENCY for OMAP5 ES2.0, and DRA7 ES1.0 */
static const struct dpll_params mpu_dpll_params_1ghz[NUM_SYS_CLKS] = {
    {250, 2, 1, 1, -1, -1, -1, -1, -1, -1, -1}, /* 12 MHz */
```

```

{500, 9, 1, 1, -1, -1, -1, -1, -1, -1, -1, -1}, /* 20 MHz */
{119, 1, 1, 1, -1, -1, -1, -1, -1, -1, -1, -1}, /* 16.8 MHz */
{625, 11, 1, 1, -1, -1, -1, -1, -1, -1, -1, -1}, /* 19.2 MHz */
{500, 12, 1, 1, -1, -1, -1, -1, -1, -1, -1, -1}, /* 26 MHz */
{-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1}, /* 27 MHz */
{625, 23, 1, 1, -1, -1, -1, -1, -1, -1, -1, -1}, /* 38.4 MHz */
};

```

NOTE: Refer to TRM and DM for max frequency limits and special cases. For example MPU_DPLL needs DCC bit to be enabled when frequency is greater than 1.4GHz

If you add a new table then update the pointer in below data structure:

```

struct dpll5 dra7xx_dpll5 = {
    .mpu = mpu_dpll_params_1ghz,
    ::::
};

```

How to change the frequency of remote cores (IVA,DSP,GPU) in kernel from u-boot?

The steps described are for two-stage boot. The below steps aren't valid for single stage boot. For single stage boot, changes must be directly made on the DTB.

- V2016.05, v2017.01,

```
ft_cpu_setup() in arch/arm/cpu/armv7/omap5/fdt.c
```

The dtb file will be updated runtime to reflect the new frequency (will not modify the dtb file but only the copy in memory will be updated) and when kernel boots it reads the updated version of the dtb in memory and configures clock rates accordingly.

How do I debug/ get more debug information for MMC related issues in u-boot?

MMC framework has a detailed tracing capability implemented which can be enabled by adding configuration option CONFIG_MMC_TRACE to your configuration file. This option enables low-level prints in mmc core driver that contains information of

- every command sent across the lines
- command the response
- card status

By reviewing the logs you'll be able to identify the command that resulted in the exact failure or the card status when the failure happened.

Kernel

How do I control MPU OPP (frequency and voltage)?

There's a framework to manage MPU OPP (frequency and voltage). Refer to [DRA7xx_PM_DVFS_User_Guide](http://processors.wiki.ti.com/index.php/GLSDK_DRA7xx_PM_DVFS_User_Guide) (http://processors.wiki.ti.com/index.php/GLSDK_DRA7xx_PM_DVFS_User_Guide) for details.

Add lower frequency OPP support for (<1GHz) J6Eco (dra72x) variants

Add new operatig points in arch/arm/boot/dts/dra72x.dtsi and delete 1GHz <syntaxhighlight lang="c"> cpu0: cpu@0 { device_type = "cpu"; compatible = "arm,cortex-a15"; reg = <0>;

operating-points = < /* kHz uV */

```

600000 1100000
800000 1100000

```

>; </syntaxhighlight>

How do I debug / get additional debug information for MMC/SD/SDIO issues?

MMC framework has a detailed tracing capability implemented which can be enabled through kenrel configuration option MMC debugging (CONFIG_MMC_DEBUG).

```

-> Device Drivers
-> MMC/SD/SDIO card support (MMC [=y])
[ ] MMC debugging

```

This option enables low-level prints in mmc core driver that contains information of

- every command sent across the lines
- command the response
- card status

By reviewing the logs you'll be able to identify the command that resulted in the exact failure or the card status when the failure happened.

- **Note:**If you don't see any additional logs on the console then you may do either of the following:
- Set loglevel=8 in kernel bootargs, this will print all logs to console.
- At kernel prompt run "dmesg", this will print all logs to console
- Also set log_buf_len=2M in boot args to avoid over-writing the log buffer. If log-buffer is set to default then you may lose some prints.

How do I test uart data transfer?

Internal loop back within the controller

Testcase 1: Data transfer with error and integrity check using serialcheck utility on uart3 instance (usually UART(x) in HW corresponds to tty\$(x-1) in SW)

Download serialcheck utility from [File:Serialcheck.zip](#).

Open two terminals on target, using "telnet <target ip addr>" and login as "root"

On Terminal #1 (Rx) 1. Run serialcheck receive mode,

1. serialcheck -d /dev/ttyS2 -b 115200 -k -m r -f /boot/dra7-evm.dtb -l 2 (replace dra7-evm.dtb with any file to transfer)

Sample output

```
root@dra7xx-evm:~# serialcheck -d /dev/ttyS2 -b 115200 -k -m r -f /boot/dra7-evm.dtb -l 2
Needed 1131 reads 0 writes loops 2 / 2
cts: 0 dsr: 0 rng: 0 dcd: 0 rx: 217314 tx: 217314 frame 0 ovr 0 par: 0 brk: 0 buf_ovrr: 0
root@dra7xx-evm:~#
```

on Terminal #2 (Tx) 2. Run serialcheck in transfer mode,

1. serialcheck -d /dev/ttyS9 -b 115200 -k -m t -f /boot/dra7-evm.dtb -l 2

Sample out

```
root@dra7xx-evm:~# serialcheck -d /dev/ttyS2 -b 115200 -k -m t -f /boot/dra7-evm.dtb -l 2
Needed 0 reads 1 writes loops 2 / 2
cts: 0 dsr: 0 rng: 0 dcd: 0 rx: 215040 tx: 215127 frame 0 ovr 0 par: 0 brk: 0 buf_ovrr: 0
root@dra7xx-evm:~#
```

In case of error (intentionally provided wrong reference file - notice am57xx-evm.dtb on the receive side) sample output

```
root@dra7xx-evm:~# serialcheck -d /dev/ttyS2 -b 115200 -k -m r -f /boot/am57xx-evm.dtb -l 2
Needed 1051 reads 0 writes 0h oh, inconsistency at pos 6 (0x6).

Original sample:
00000000: d0 0d fe ed 00 01 89 ff 00 00 00 38 00 01 7a d4 .....8..z.
00000010: 00 00 00 28 00 00 00 11 00 00 00 10 00 00 00 00 ...(.
00000020: 00 00 0f 2b 00 01 7a 9c 00 00 00 00 00 00 00 00 ...+..z.....

Received sample:
00000000: d0 0d fe ed 00 01 a8 71 00 00 00 38 00 01 97 ec .....q...8...
00000010: 00 00 00 28 00 00 00 11 00 00 00 10 00 00 00 00 ...(.
00000020: 00 00 10 85 00 01 97 b4 00 00 00 00 00 00 00 00 .....

loops 1 / 2

cts: 0 dsr: 0 rng: 0 dcd: 0 rx: 100896 tx: 98304 frame 0 ovr 0 par: 0 brk: 0 buf_ovrr: 0
root@dra7xx-evm:~#
```

How do I measure time taken to execute code (timestamp) better than printk time?

One can use gettimeofday() to get the time in nano second resolution. Take a timestamp once before and once after the execution of code block / api of interest. Find the difference to get the time taken in ns. Note: Resolution / accuracy depends on timer used by kernel. This assumes a higher resolution timer available on the system.

```
+ struct timespec ts1, ts2;
+ unsigned long dt = 0;

+ /* Start timestamp */
+ gettimeofday(&ts1);

+ /* code or api for which time of execution is being measured*/
+ code_of_interest();

+ /* Stop timestamp */
+ gettimeofday(&ts2);

+ dt = (((ts2.tv_sec - ts1.tv_sec) * NSEC_PER_SEC + (ts2.tv_nsec - ts1.tv_nsec)) / NSEC_PER_USEC);
+ pr_err("Time to complete execution | %lu us |\n", dt);
```

Multimedia

How do I fix the Yocto build issue while trying to build IPUMM component ?

The IPUMM git repository went through a full repository rewrite. Although this doesn't affect any functionality, it breaks the releases that have already been made. It is possible that customers who are building the Processor SDK (all variants) will be impacted.

A fix has been pushed into the latest meta-ti on both krogoth and morty branches, but it will definitely affect customers building the previous releases.

In order to help customers to figure out the commit ID change, a table mapping the older commit to the newer commit is available in this attachment : [ipumm-commit-id-differences.pdf \(http://processors.wiki.ti.com/index.php/File:ipumm-commit-id-differences.pdf\)](http://processors.wiki.ti.com/index.php/File:ipumm-commit-id-differences.pdf)

How do I fix the issue with fetching IPUMM sources in the PSDKLA 3.03 or older installers

The root cause of the problem is exact the same as the previous section. If you see an error like this:

```
Fetching project repo/u-boot
error: Cannot fetch ivimm/ipumm (GitError: ivimm/ipumm update-ref: fatal: fd441443a4289c801a0c8d9f00b6966f7fe3476c^0: not a valid SHA1)
```

Then please refer to the PDF in the previous question and update the manifest file vi .repo/manifests/processor-sdk_<version>.xml with the new SHA1 and then attempt to repo sync.

Additional FAQs

Link to additional FAQs | GLSDK FAQs (http://processors.wiki.ti.com/index.php?title=GLSDK_FAQs)

Keystone=

1. switchcategory:MultiCore=

■ For technical support on MultiCore devices, please post your questions in the C6000 MultiCore Forum

■ For questions related to the BIOS MultiCore SDK (MCSDK), please use the BIOS Forum

Please post only comments related to the article **Processor SDK Linux Automotive FAQ** here.

C2000=For technical support on the C2000 please post your questions on The C2000 Forum. Please post only comments about the article

DaVinci=For technical support on the DaVinciplease post your questions on The DaVinci Forum. Please post only comments about the article

MSP430=For technical support on MSP430 please post your questions on The MSP430 Forum. Please post only comments about the article

OMAP35x=For technical support on OMAP please post your questions on The OMAP Forum. Please post only comments about the article

OMAPL1=For technical support on OMAP please post your questions on The OMAP Forum. Please post only comments about the article

MAVRK=For technical support on MAVRK please post your questions on The MAVRK Toolbox Forum. Please post only comments about the article

For technical support please post your questions at <http://e2e.ti.com>. Please post only comments about the article **Processor SDK Linux Automotive FAQ** here.

}}

Links



[Amplifiers & Linear Audio](#)

[Broadband RF/IF & Digital Radio](#)

[Clocks & Timers](#)

[Data Converters](#)

[DLP & MEMS High-Reliability Interface](#)

[Logic](#)

[Power Management](#)

[Processors](#)

■ [ARM Processors](#)

■ [Digital Signal Processors \(DSP\)](#)

■ [Microcontrollers \(MCU\)](#)

■ [OMAP Applications Processors](#)

[Switches & Multiplexers](#)

[Temperature Sensors & Control ICs](#)

[Wireless Connectivity](#)

Retrieved from "https://processors.wiki.ti.com/index.php?title=Processor_SDK_Linux_Automotive_FAQ&oldid=234681"

This page was last edited on 24 May 2018, at 05:36.
Content is available under [Creative Commons Attribution-ShareAlike](#) unless otherwise noted.